MSAGLJS: Graph Layout, Edge Routing, and Tiling for the Web

Anonymous author(s)
Anonymous affiliation(s)

Abstract

We present MSAGLJS, an open-source TypeScript library for graph visualization in web browsers, distributed as a set of NPM packages[1]. It provides a set of layout algorithms, a set of edge routing algorithms, and two renderers that run in an Internet browser: one based on SVG and another based on WebGL via the deck.gl framework. In this paper we concentrate on two features of the latter renderer.

First, a *tiling scheme* that turns a large graph into a navigable map: at every zoom level the labels of the highest-ranked nodes remain readable, just as the names of major geographical features stay readable on digital maps—making the browsing experience familiar to anyone used to online maps. Second, a *corridor routing* method that routes each edge along the shortest path in homotopy class around the node obstacles, based on the Constrained Delaunay Triangulation. The whole pipeline runs client-side, without a dedicated server, on a phone or a laptop. In this configuration the WebGL renderer interacts smoothly with graphs of up to 10 000 nodes and 40 000 edges.Re-run benchmarks to confirm the maximum graph size; current 10 000 / 40 000 figures are placeholders.

MSAGLJS is available at `https://github.com/microsoft/msagljs`.

2012 ACM Subject Classification Human-centered computing → Graph drawings; Theory of computation → Computational geometry

Keywords and phrases Graph visualization, edge routing, Constrained Delaunay Triangulation, tiling, WebGL

Category Track 2, Long Paper

Generative AI Declaration Generative AI was used for editorial assistance in the preparation of this article.

1 Introduction

Visualizing graphs in web browsers without a dedicated server presents unique challenges: the runtime is single-threaded, the per-tab JavaScript heap is capped at roughly 4 GB,[2] and users expect the smooth pan-and-zoom experience familiar from online maps. MSAGLJS[3] is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme for level-of-detail rendering.

MSAGLJS is consumed as a set of NPM packages and renders graphs using WebGL through the deck.gl framework [4]. It supports the layered Sugiyama scheme [41], Pivot MDS [15], and IPSep-CoLa [21], with edges routed around node obstacles. Throughout this paper our typical example is a social network laid out by IPSep-CoLa; the methods we describe, however, are agnostic to the layout algorithm and apply to any node placement in which the nodes do not overlap. The rendering pipeline builds a hierarchy of tiles—analogous to map tiles—so that at any zoom level only a bounded number of entities are drawn. Tiles

[1] `@msagl/core`, `@msagl/drawing`, `@msagl/parser`, `@msagl/renderer-webgl` — all available on `https://www.npmjs.com/`.

[2] In current Chrome, V8 with pointer compression caps the per-tab JavaScript heap at approximately 4 GB (2^{32} bytes).

[3] `https://github.com/microsoft/msagljs`

34th International Symposium on Graph Drawing and Network Visualization (GD 2026).
Editors: TBD; Article No. 0; pp. 0:1–0:11
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

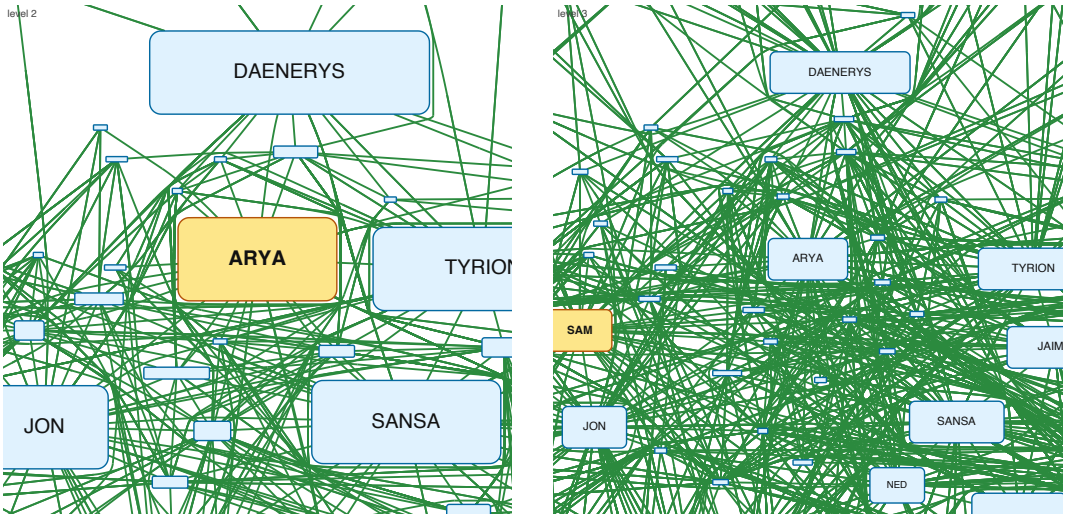

Figure 1 Overview of the per-level-graph tiling scheme presented in Section 4, on the Game of Thrones graph. Two adjacent pyramid levels are shown, cropped to the same window; the coarser level is on the left, the finer one on the right. For each level a new graph is assembled from a PageRank-ranked subset of the nodes, and the top-ranked landmark, highlighted in amber, enters that level enlarged by a factor of two in each dimension. Remaining survivors, shown in light blue, are kept at a node-specific scale $s_v \in [0.25, 2]$ that shrinks with rank so that they do not overlap. Edges are rerouted per level with a sleeve-hint A* built from the finer-level route, which keeps the coarse shape close to the fine one and yields smooth transitions as the user pans and zooms.

are delivered to the browser through the standard `TileLayer` of the `@deck.gl/geo-layers` package, giving smooth pan and zoom. Live demos are available online: a WebGL renderer for large graphs[4] and an SVG renderer for smaller, editable graphs[5].

This paper makes two main contributions:

1. A *tiling scheme* for large graph visualization that builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.
2. A *corridor routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation, using per-source batched Dijkstra trees followed by the classical funnel algorithm to produce shortest-path geodesics in homotopy classes.

2 Related Work

2.0.0.1 Graph visualization tools.

Graphviz [25, 6] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [9]; it does not support tiling or WebGL rendering. Our library builds on the earlier MSAGL desktop layout engine [37] that did not have the features described here. On the web, Sigma.js [10] and Cytoscape.js [23] provide interactive graph rendering, but rely on straight-line or generic spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles.

[4] `https://microsoft.github.io/msagljs/renderer-webgl/index.html`
[5] `https://microsoft.github.io/msagljs/renderer-svg/index.html`

ReGraph [8] uses WebGL to render large graphs with straight-line edges and supports interactive cluster expansion but not tiling. Cosmograph [3] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [36] supports tiling and level-of-detail for large graphs but runs only on Windows and does not optimize edge routes. Cornac [39] handles huge graphs with tiles and edge bundling [34, 32] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [13], which organizes nodes into a hierarchical cluster tree, and topological fisheye views [24], which distort a multilevel layout [27, 33] under the focus; neither targets browser-side tile pyramids or per-level edge rerouting. Hierarchical edge bundling [31] and force-directed bundling [32] are complementary clutter-reduction techniques applied *after* edges have been routed.

2.0.0.2 Shortest paths and edge routing.

Our corridor algorithm walks the dual of a Constrained Delaunay Triangulation [40] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [35, 16, 26, 5]. The optimal algorithm for Euclidean shortest paths among polygonal obstacles [30] and the recent $O(m \log^{2/3} n)$ single-source shortest-path algorithm that breaks the sorting barrier [20] are remarkable theoretical results, but they are not practical: both build elaborate global data structures that we avoid. Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [18] but requires a simple containing polygon. yWorks [12] offers "Organic edge routing" based on force-directed simulation. The approach of Dwyer and Nachmanson [22] routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid framework of Wybrow, Marriott and Stuckey performs obstacle-avoiding connector routing with incremental updates [42]. Our corridor method eliminates the spanner entirely and routes directly on the CDT dual graph, and is, to our knowledge, the first routing algorithm integrated with a browser-side tile pyramid that reroutes edges per level.

3 System Architecture

MSAGLJS is organized as a monorepo with five NPM packages:

- `@msagl/core` — Graph data structures, layout algorithms (Sugiyama, MDS, IPSep-CoLa), edge routing, CDT, tiling.
- `@msagl/drawing` — Visual attributes (colors, shapes, labels) bridging the graph model and renderers.
- `@msagl/parser` — Parsers for DOT and JSON graph formats.
- `@msagl/renderer-svg` — SVG-based renderer for small graphs.
- `@msagl/renderer-webgl` — WebGL renderer using deck.gl [4], with tiling support.

The layout pipeline proceeds in three phases:

1. **Layout.** Node positions are computed by the chosen algorithm. For large undirected graphs the default is force-directed layout (IPsepCola) initialized with Pivot MDS [15]; for directed graphs, Sugiyama layout is used.
2. **Edge routing.** Edges are routed around node obstacles. The routing mode is configurable; this paper focuses on *corridor routing* (Section 5).
3. **Tiling.** A tile hierarchy is built for level-of-detail rendering (Section 4).

The WebGL renderer (Section 7) consumes the tile hierarchy and renders the appropriate level of detail based on the current viewport.

4 Tiling

To visualize large graphs interactively, MSAGLJS builds a pyramid of tiles analogous to those used in online maps. Levels are numbered $0, 1, \ldots, Z$ from coarsest to finest; the finest level Z shows the full graph produced by the layout and the corridor router presented in Section 5. At any zoom only the few tiles of the current level that intersect the viewport are fetched and drawn, so rendering cost is proportional to the number of entities in those visible tiles, not to the size of the graph. The construction proceeds in three phases.

4.1 Tile Hierarchy Construction

A tile is a pair $(\mathit{rect}, \mathit{tiledata})$, where *tiledata* is a set of *tile elements*: nodes, edge labels, edge arrowheads, and *edge clips*. An edge clip is a pair (e, p), with p a continuous piece of the edge curve c_e confined to *rect*.

The single tile at level 0 is obtained by clipping every element to the graph's bounding box. To build level $z+1$ from level z, each tile is split into four equal sub-tiles; nodes, labels, and arrowheads are assigned to sub-tiles by bounding-box intersection, and each edge clip is cut at the tile midlines, producing sub-clips confined to individual sub-tiles. We maintain invariant $\mathcal{F}$: (a) every clip stays inside its tile's rectangle, crossing the boundary only at its endpoints; (b) the union of all clips for edge e inside a tile equals $c_e \cap \mathit{rect}$. Subdivision of a level stops as soon as either of the following holds: (i) every tile at that level already contains at most $\mathcal{C}$ elements; or (ii) the tile width and height have both dropped below ten times the average node width and height, respectively. The first condition uses a default $\mathcal{C} = 500$, so a tile that is already light enough is not splitg. The second condition prevents producing tiles smaller than the nodes they display.

4.2 Per-Level Graph and PageRank-Guided Filtering

To keep coarser levels readable, each level $z < Z$ is assigned its own *level graph* G_z: a subset $V_z \subseteq V$ of active nodes together with every edge of G whose endpoints both lie in V_z. We first compute PageRank [38] on G once and sort nodes in decreasing rank.

Starting from $V_Z = V$, we build V_{z-1} greedily. Let $r(v)$ be the rank of v in V_z, so that rank 1 denotes the highest PageRank in V_z. The top-ranked node $v^\star$ is always accepted and *enlarged by a factor of two in each dimension*; lower-ranked nodes inherit a scale that is monotonically non-increasing in rank. We walk the top half of V_z in rank order and, for each candidate v, pick the largest scale $s_v \in [s_{\min}, 2]$, decreasing geometrically, whose inflated bounding box, padded by a margin $m = 3p + p_{\text{extra}}$ matching the obstacle inflation used by the router, does not intersect any previously accepted box. Scales below 1 are allowed, down to $s_{\min} = 0.25$, so that crowded lower-ranked nodes can still be shown at a reduced size rather than being dropped; a candidate whose box overlaps even at $s_{\min}$ is skipped. The accepted nodes form V_{z-1} and the chosen scales are stored per level. This produces the intended map-like effect: a small number of big, highly ranked landmarks at the coarsest zoom, and progressively more detail as the user zooms in, with no lower-ranked node ever drawn larger than a higher-ranked one.

4.3 Stable Per-Level Rerouting with a Routing Hint

Because G_{z-1} has fewer obstacles than G_z, and different ones on account of the enlargement of $v^\star$, edges of G_{z-1} must be rerouted: the finest-level curves would otherwise pierce the

enlarged landmark boxes. We rerun the corridor router (Section 5) on $G_z \setminus G_{z-1}$'s active edges, but with two modifications that secure visual stability during zoom.

4.3.0.1 Per-level CDT.

For level $z-1$ we build a dedicated CDT from the inflated polylines of V_{z-1} only, using each node's level-$z{-}1$ scale. The highest-ranked landmark thus enters the triangulation as an obstacle that is twice its finest-level size in each dimension, so routes naturally bend around it.

4.3.0.2 Routing hint.

For every edge $e \in E_{z-1}$ we already have a route c_e computed at the finer level z. We sample c_e, locate the CDT triangles it visits at level $z-1$, and expand this sequence by one neighbor hop to obtain a triangle *sleeve* S_e. Instead of the per-source Dijkstra trees described in Section 5.4, which are efficient when many edges share a source but ignore the previous level's shape, we run a per-edge **A*** on the dual of the level-$z{-}1$ CDT, *restricted to* S_e, with the source and target obstacle triangles always admissible. The hint keeps the coarse-level route topologically and geometrically close to the fine-level one, so that as the user zooms out an edge deforms smoothly rather than jumping to a different side of a landmark. If the masked A* fails, for instance when the target now lies inside the inflated landmark, we fall back to unconstrained A* on the whole dual, and finally to A* with the obstacle-interior guard disabled; all three tiers share the same pre-allocated open-set arrays.

To summarize the division of labor: **Dijkstra trees**, Section 5.4, drive the *finest-level* routing, where there is no hint and many edges share sources; **A*** drives the *per-level rerouting* for $z < Z$, where each edge carries its own sleeve hint from the finer level and independent searches are required. Figure 1 on the first page shows two adjacent pyramid levels built by this procedure. Once E_{z-1} has been rerouted, the clip invariant $\mathcal{F}$ is reestablished by recomputing the clips of the affected edges inside every level-$(z{-}1)$ tile.

5 Corridor Routing on the CDT

We describe an edge routing method that routes edges directly on the dual graph of a Constrained Delaunay Triangulation, without requiring a spanner graph. The method finds a *corridor*—a sequence of adjacent CDT triangles from source to target—and applies the funnel algorithm to produce the shortest path within it.

5.1 CDT Construction and the Dual Graph

Given the graph layout, each node is covered by a padded obstacle polygon. We compute a Constrained Delaunay Triangulation $\mathcal{T}$ on the set $\mathcal{O}$ of these obstacle polygons [17, 19]. The *dual graph* D of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing a side. A triangle is *free-space* if it is not contained in any obstacle interior. Each triangle of an A* (or Dijkstra) path is entered through one shared edge and left through another; the cost of the step taken inside the triangle is the Euclidean distance between the midpoints of those two edges. For the source triangle, which has no entering edge, the centroid is used as the entry point.

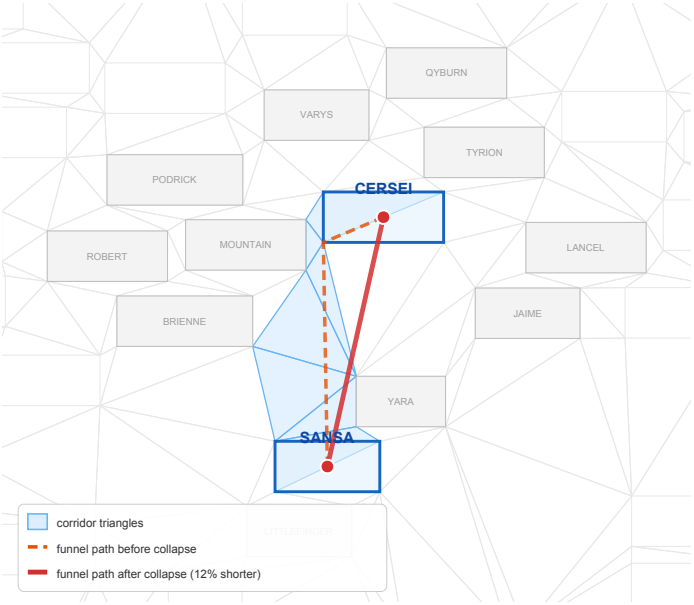

Figure 2 Effect of vertex collapse on the CERSEI–SANSA edge. Light blue: corridor triangles. Orange dashed: funnel path before collapse (3 waypoints). Red solid: funnel path after collapse (2 waypoints, 12% shorter). Collapsing obstacle vertices to the node center widens the funnel opening and removes the detour.

5.2 A* Search on the Dual Graph

For each graph edge from source s to target t, we locate the containing triangles and run A* [28] on the dual graph D. The heuristic is the straight-line distance from the current centroid to t. Triangles inside obstacles (other than the source and target obstacles) are excluded from the search. The resulting corridor is passed to the funnel algorithm [16, 29], which computes the shortest path through the corridor's sequence of diagonals.

5.3 Vertex Collapse at Source and Target

The corridor passes through triangles whose vertices lie on the padded obstacle boundaries. When these vertices appear as diagonal endpoints, the funnel algorithm produces sharp zigzag turns near each endpoint. We eliminate these artifacts by *collapsing* obstacle vertices: any diagonal endpoint belonging to the source obstacle is replaced with the source center s, and similarly for the target. Degenerate diagonals (both endpoints collapsed to the same point) are removed. The left/right orientation of each diagonal is preserved from the uncollapsed geometry.

After collapse, the funnel sees a wide opening at each endpoint and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves.

5.4 Batched Routing with Dijkstra Trees

When a node has many incident edges, running independent A* searches is wasteful. We group edges by source node and run a single Dijkstra computation per source on the dual graph, expanding until all target triangles are reached. Corridors are recovered by following

parent pointers. This reduces the number of searches from m (edges) to at most n (nodes). On the Game of Thrones social network (407 nodes, 2 639 edges, 331 distinct sources), the Dijkstra tree approach reduces routing time by 30%.

6 The Portal Graph

The batched Dijkstra approach of Section 5.4 still requires exploring much of the CDT for each source node. To enable faster queries, we introduce the *portal graph*—an abstraction that separates the free-space routing problem from the local obstacle traversal.

6.1 Portal Triangles

For each node obstacle, we identify its *portal triangles*: the free-space CDT triangles that share an edge with the obstacle boundary. Formally, a triangle t is a portal of obstacle O if t is free-space and at least two of its three CDT sites belong to O's boundary polyline. Portals serve as entry and exit points: any shortest path from the interior of O to free space must pass through a portal triangle.

6.2 Structure of the Portal Graph

The portal graph G_P is defined as follows:

- **Nodes.** For each graph node v, the center point c_v and all portal triangles $\{p_1^v, p_2^v, \ldots\}$ of v's obstacle are nodes of G_P.
- **Interior edges.** Each center c_v is connected to every portal p_i^v of the same obstacle, with weight equal to the BFS distance through the obstacle's interior triangles.
- **Free-space edges.** Portal triangles of different obstacles are connected through the free-space CDT dual graph.

To route a graph edge from node w to node u, we find the shortest path in G_P from c_w to c_u. This path decomposes into three segments:

1. An *interior path* from c_w through w's obstacle to some portal p_i^w.
2. A *free-space path* from p_i^w to some portal p_j^u of u.
3. An *interior path* from p_j^u through u's obstacle to c_u.

The free-space segment (2) is the bottleneck: it requires a shortest-path query on the free-space dual graph, which may contain thousands of triangles. We answer these queries with the batched Dijkstra trees of Section 5.4.

7 WebGL Rendering

The WebGL renderer uses deck.gl [4], a GPU-accelerated framework for large-scale data visualization. The renderer sets up an `OrthographicView` (no perspective distortion) and a `TileLayer` that dynamically loads tile data based on the current viewport.

7.0.0.1 Tile resolution mapping.

The root tile size is rounded up to the nearest power of two. A *start zoom* level is computed so that the root tile maps to the standard 512-pixel tile size of deck.gl. The `TileLayer`'s `getTileData` callback translates deck.gl tile coordinates (x, y, z) to MSAGLJS tile map coordinates, fetching the precomputed tile data.

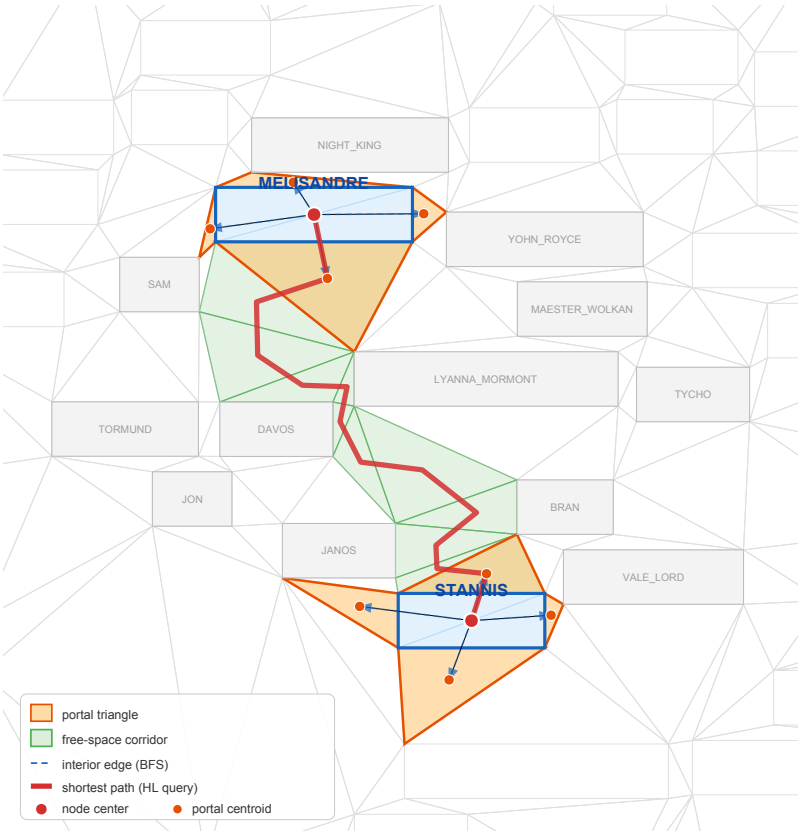

Figure 3 The portal graph for the MELISANDRE–STANNIS edge on the Game of Thrones social network. Orange triangles are portals—free-space CDT triangles adjacent to the obstacle boundary. Blue dashed arrows show interior edges from node centers to portal centroids. Green-shaded triangles form the free-space corridor found by the batched Dijkstra search. The thick red polyline shows the shortest path $c_{\text{MEL}} \to p_i \to \cdots \to p_j \to c_{\text{STAN}}$. Nearby node obstacles are shown in gray.

7.0.0.2 Rendering layers.

Each tile is rendered as a composite deck.gl layer containing:

- A **GeometryLayer** for node shapes (rectangles, ovals, diamonds).
- A **TextLayer** for node labels.
- A custom **CurveLayer** for edge paths, supporting cubic Bézier curves, arcs, and line segments via GPU-side interpolation. Curve tessellation resolution scales with zoom: at zoom level z, the resolution is 2^{z-2} segments per curve span.
- An **IconLayer** for edge arrowheads.
- A **TextLayer** for edge labels.

7.0.0.3 Style-based filtering.

A declarative style specification allows per-layer control of visibility ranges (`minZoom`, `maxZoom`), colors, opacity, sizes, and filters. Styles can reference the node's PageRank to, e.g., show labels only for important nodes at medium zoom levels.

Graph	Nodes	Edges	CDT △	Free △	Portals	Dijkstra (ms)
GoT [14]	407	2 639	3 258	2 444	1 628	277
b103 [2]	944	2 438	7 554	5 666	3 776	754
b100 [1]	1 463	5 806	11 706	8 780	5 852	2 120
composers [11]	3 405	13 832	27 242	20 432	13 620	9 335
Gnutella04 [7]	10 876	39 994	87 010	65 258	43 504	93 760

Table 1 Corridor routing benchmarks. Dijkstra: per-source Dijkstra trees on the CDT dual.

7.0.0.4 Interaction.

The renderer supports pan, zoom, hover highlighting, and click selection. Hovering an edge highlights it and its incident nodes. The `zoomTo` API animates the viewport to a target rectangle with smooth transitions.

8 Experimental Results

We evaluate the corridor routing method on benchmark graphs from the Game of Thrones social network [14], the GD 2011 contest [11], the Graphviz test suite [2, 1], and the Gnutella P2P network [7]. All experiments run in Node.js on a laptop with an Apple M1 processor. Layout uses Pivot MDS; timings include CDT construction, routing, and curve smoothing.

Table 1 reports the performance of the Dijkstra-tree corridor router (Section 5.4). For each graph, we report the number of CDT triangles, free-space triangles, the total number of portal triangles across all nodes, and the total routing time.

The Dijkstra-tree approach scales to graphs with tens of thousands of edges. Routing the full Gnutella04 graph (10 876 nodes, 39 994 edges) on an Apple M1 laptop takes about 94 s, which is amortized in interactive use because tile pyramids cache the routes computed at the finest level and rerun only the per-level A* described in Section 4 on coarser zooms.

9 Conclusion

We presented MSAGLJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The corridor routing method routes edges directly on the CDT dual graph using a portal graph abstraction and the funnel algorithm. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge simplification. The WebGL renderer, built on deck.gl, provides smooth pan-and-zoom interaction.

MSAGLJS is available at `https://github.com/microsoft/msagljs` under the MIT license.

References

1 b100. `https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b100.dot`.

2 b103. `https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b103.dot`.

3 Cosmograph. `https://cosmograph.app`.

4 deck.gl: Large-scale WebGL-powered data visualization. `https://deck.gl/`.

5 Funnel algorithm. `https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf`.

6 Graphviz. `http://www.graphviz.org/`.

7 p2p-gnutella04. `https://snap.stanford.edu/data/p2p-Gnutella04.html`.

8 Regraph. https://cambridge-intelligence.com/regraph/.

9 sfdp. https://graphviz.org/docs/layouts/sfdp/.

10 Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. https://www.sigmajs.org/.

11 Skewed. http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml.

12 yworks. https://yworks.com/products/yed.

13 James Abello, Frank Van Ham, and Neeraj Krishnan. ASK-GraphView: A large scale graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*, volume 12, pages 669–676. IEEE, 2006.

14 Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages 201–225. Routledge, 2018.

15 Ulrik Brandes and Christian Pich. Eigensolver methods for progressive multidimensional scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe, Germany, September 18-20, 2006. Revised Papers 14*, pages 42–53. Springer, 2007.

16 Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.

17 Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennyka Nauk*, 7(1):793–800, 1934.

18 David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD'97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.

19 Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008.

20 Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025. https://arxiv.org/abs/2504.17033.

21 Tim Dwyer, Yehuda Koren, and Kim Marriott. IPSep-CoLa: An incremental procedure for separation constraint layout of graphs. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):821–828, 2006. doi:10.1109/TVCG.2006.156.

22 Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing: 17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised Papers 17*, pages 147–158. Springer, 2010.

23 Max Franz, Christian T. Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D. Bader. Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–311, 2016.

24 Emden R. Gansner, Yehuda Koren, and Stephen North. Topological fisheye views for visualizing large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468, 2005.

25 Emden R. Gansner and Stephen C. North. An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233, 2000.

26 Leonidas Guibas, John Hershberger, Daniel Leven, Micha Sharir, and Robert E. Tarjan. Linear-time algorithms for visibility and shortest path problems inside triangulated simple polygons. *Algorithmica*, 2(1–4):209–233, 1987.

27 David Harel and Yehuda Koren. A fast multi-scale method for drawing large graphs. In *Journal of Graph Algorithms and Applications*, volume 6, pages 179–202, 2002.

28 Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.

29 John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy class. *Computational geometry*, 4(2):63–97, 1994.

30 John Hershberger and Subhash Suri. An optimal algorithm for Euclidean shortest paths in the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999.

31 Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.

32 Danny Holten and Jarke J. Van Wijk. Force-directed edge bundling for graph visualization. In *Computer Graphics Forum*, volume 28, pages 983–990. Wiley Online Library, 2009.

33 Yifan Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–71, 2005.

34 Christophe Hurter, Ozan Ersoy, and Alexandru Telea. Graph bundling by kernel density estimation. In *Computer graphics forum*, volume 31, pages 865–874. Wiley Online Library, 2012.

35 D. T. Lee and Franco P. Preparata. Euclidean shortest paths in the presence of rectilinear barriers. *Networks*, 14(3):393–410, 1984.

36 Lev Nachmanson, Roman Prutkin, Bongshin Lee, Nathalie Henry Riche, Alexander E Holroyd, and Xiaoji Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA, September 24-26, 2015, Revised Selected Papers 23*, pages 3–15. Springer, 2015.

37 Lev Nachmanson, George G. Robertson, and Bongshin Lee. Drawing graphs with GLEE. In *Graph Drawing: 15th International Symposium, GD 2007*, pages 389–394. Springer, 2008.

38 Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999.

39 Alexandre Perrot and David Auber. Cornac: Tackling huge graph visualization with big data infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018.

40 Jonathan Richard Shewchuk. Delaunay refinement algorithms for triangular mesh generation. *Computational Geometry*, 22(1–3):21–74, 2002.

41 Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981. doi:10.1109/TSMC.1981.4308636.

42 Michael Wybrow, Kim Marriott, and Peter J. Stuckey. Orthogonal connector routing. *Lecture Notes in Computer Science*, 5849:219–231, 2010.